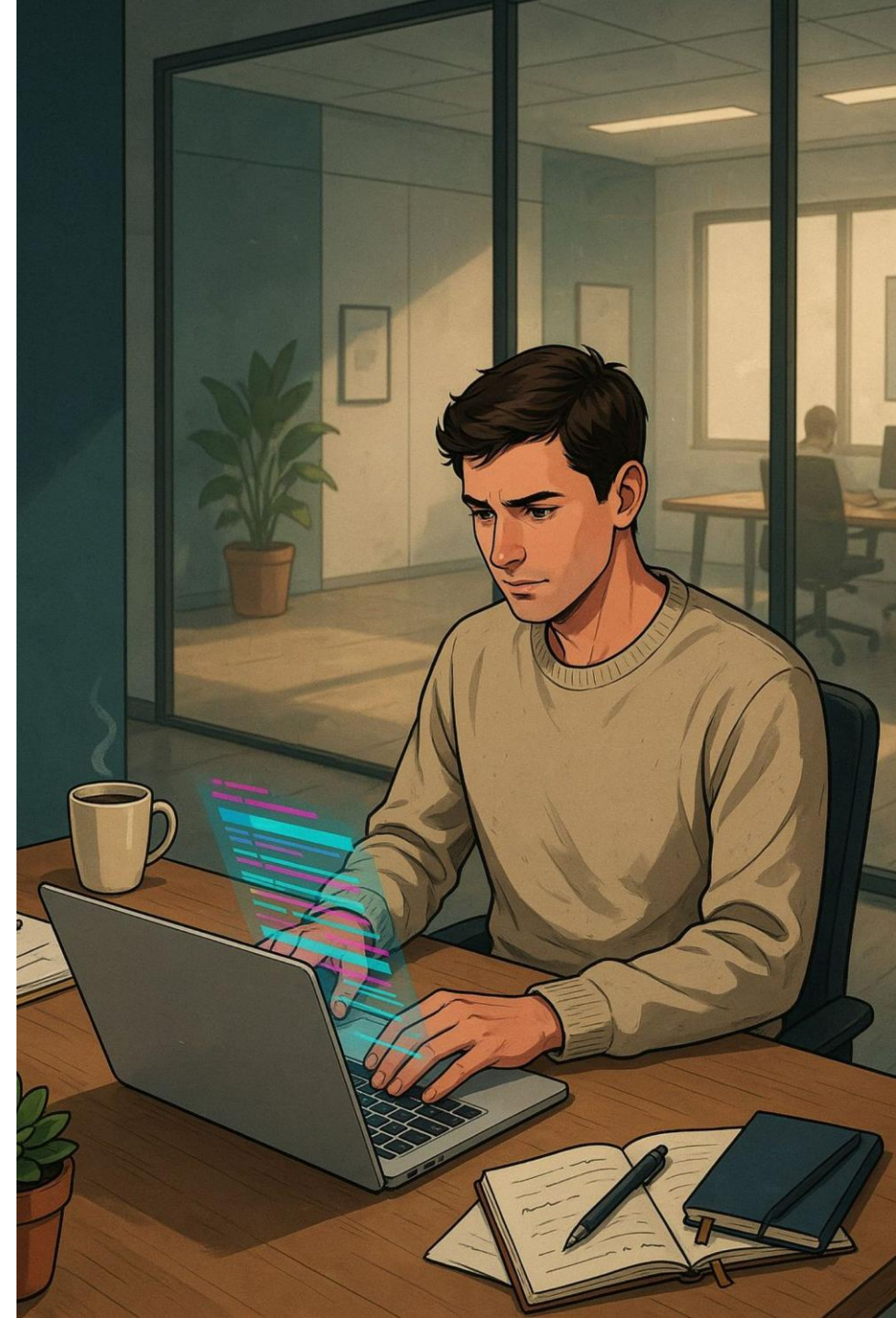


Sistem Temu Kembali Informasi

Pertemuan 2: Dari Boolean ke VSM + TF-IDF

"Kenapa Google tahu apa yang kamu maksud?"

Dalam pertemuan ini kita akan membongkar logika di balik mesin pencari modern – mulai dari model Boolean yang sederhana hingga Vector Space Model dan TF-IDF yang menjadi tulang punggung Google, Tokopedia, hingga Netflix.



Capaian Pembelajaran

Setelah menyelesaikan pertemuan ini, diharapkan mahasiswa mampu memahami dan menerapkan konsep-konsep inti sistem temu kembali informasi modern.

1

Kritik Boolean

Jelaskan mengapa model Boolean bersifat "kaku" dan tidak memadai untuk kebutuhan pencarian modern.

2

VSM + TF-IDF

Jelaskan cara kerja "otak" di balik mesin pencari: Vector Space Model dan pembobotan TF-IDF.

3

Hitung Manual

Hitung skor relevansi dokumen terhadap query secara manual menggunakan cosine similarity.

4

Praktikum Python

Implementasikan mesin pencari mini berbasis TF-IDF menggunakan scikit-learn di Google Colab.

Review: Masalah dari Pertemuan 1

Di Pertemuan 1, kita membangun pencarian sederhana berdasarkan **penghitungan kata yang cocok**. Pendekatan ini berhasil – tapi hanya sampai batas tertentu.

Skenario Nyata

Kamu mengetik "**laptop gaming murah**" di mesin pencari. Sistem menemukan 50.000 dokumen yang mengandung setidaknya satu kata tersebut.

Pertanyaannya: Dokumen mana yang ditampilkan paling atas?

Akar Masalahnya

Dengan metode hitung kata biasa, semua 50.000 dokumen itu memiliki **nilai yang sama**. Tidak ada cara untuk membedakan mana yang paling relevan.

Kita tidak butuh sekadar filter – kita butuh **SKOR** untuk memberi peringkat hasil pencarian.



Model Boolean: Filter Shopee

Definisi & Cara Kerja

Model Boolean adalah model retrieval paling klasik. Hasilnya bersifat **biner**: sebuah dokumen hanya bisa **COCOK = 1** atau **TIDAK COCOK = 0**. Tidak ada nilai di antaranya.

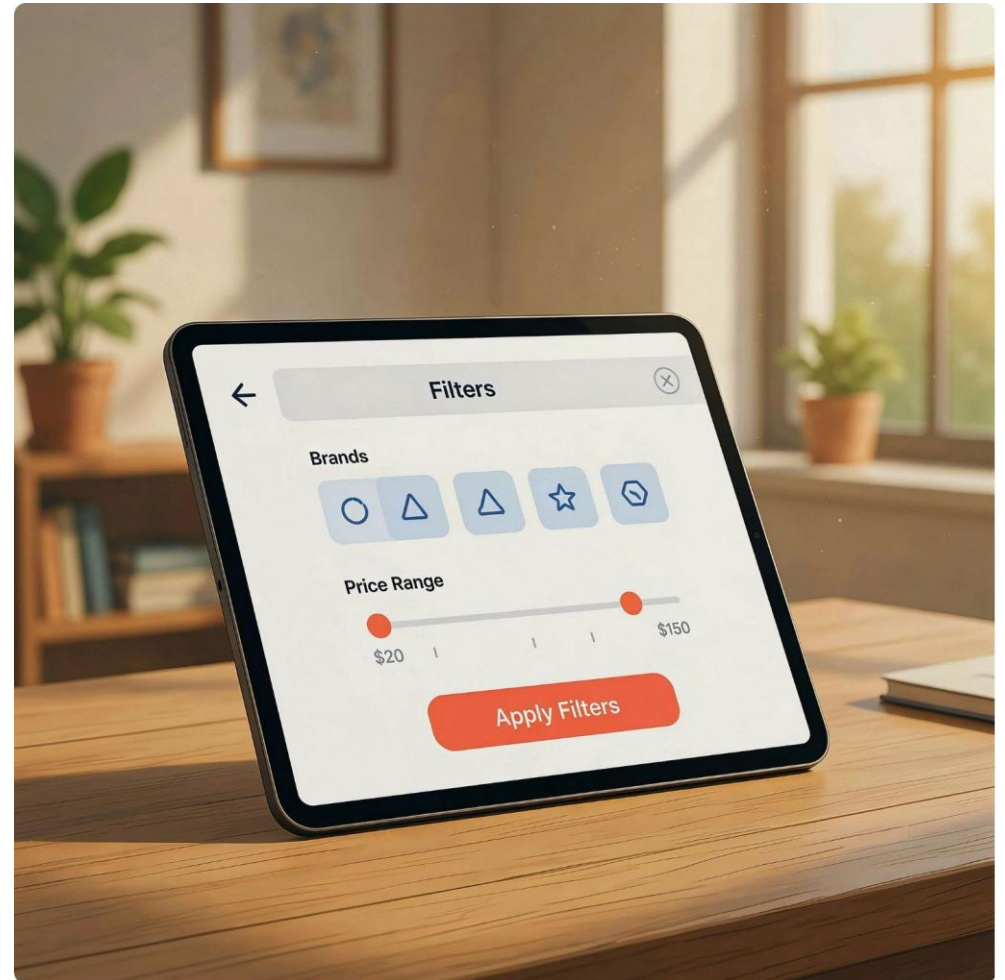
Model ini menggunakan tiga operator logika:

- **AND** – semua kata harus ada
- **OR** – salah satu kata cukup
- **NOT** – kata tertentu tidak boleh ada

Contoh Query Shopee

```
("HP" AND "Samsung") AND NOT "bekas"
```

Hasil: 120 HP Samsung baru ditemukan. Tapi HP seharga 2 juta dan 20 juta **dianggap sama relevannya** – tidak ada ranking sama sekali.



Kelemahan Model Boolean

Model Boolean sangat powerful untuk pencarian hukum atau database terstruktur – tetapi tidak dirancang untuk kebutuhan pencarian sehari-hari yang penuh ambiguitas.

Tidak Ada Ranking

Cari "**resep ayam goreng**" → 1 juta hasil muncul tanpa urutan prioritas. Pengguna harus memilah sendiri mana yang relevan.

Harus Jago Operator

Cari "**sepeda**" tanpa menambahkan **OR** "**bicycle**" → hasil bisa 0. Pengguna awam tidak tahu cara merumuskan query yang tepat.

Tidak Toleran Typo

Ketik "**jarnigan**" sebagai ganti "jaringan" → sistem langsung gagal. Tidak ada toleransi terhadap kesalahan ejaan sekecil apapun.

📌 Kesimpulan: Model Boolean cocok untuk lawyer dan analis database – tidak cocok untuk mesin pencari publik seperti Google atau marketplace modern.

Vector Space Model: Otak Netflix

Ide Dasarnya

VSM mengubah teks menjadi angka, lalu angka tersebut direpresentasikan sebagai **vektor** dalam ruang multidimensi. Prinsipnya sederhana: **semakin dekat dua vektor, semakin mirip maknanya.**

Analogi Netflix

Netflix membangun "**profil rasa**" dari setiap pengguna dan setiap film dalam bentuk vektor:

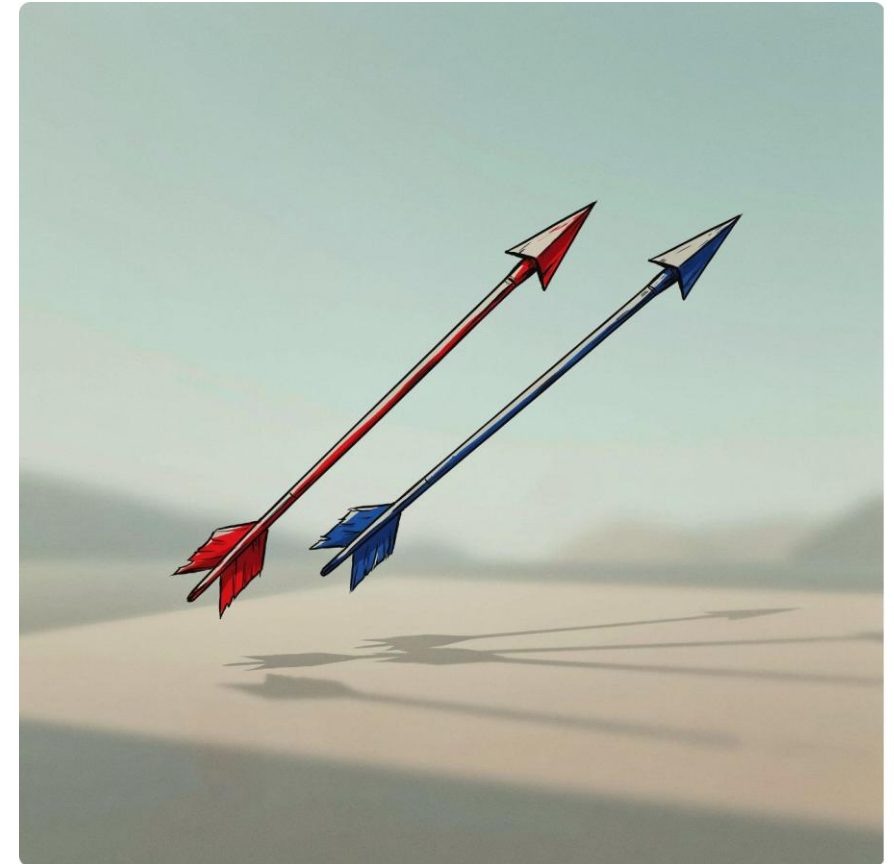
- Vektor A = Selera tontonanmu
- Vektor B = Profil film Drakor
- Vektor C = Profil film Horor

Jika sudut antara Vektor A dan B kecil → Netflix merekomendasikan Drakor terlebih dahulu.

Rumus Cosine Similarity

$$\cos(\theta) = \frac{A \cdot B}{||A|| \times ||B||}$$
$$\cos(\theta) = \frac{A \cdot B}{||A|| \times ||B||}$$

Hasil **1** = sangat mirip | Hasil **0** = berbeda total



4 Langkah Kerja VSM

Proses VSM berjalan dalam empat tahap sistematis – dari teks mentah hingga daftar hasil yang terurut berdasarkan relevansi.



Bikin Kamus

Kumpulkan semua kata unik dari seluruh dokumen dalam koleksi. Ini menjadi dimensi ruang vektor.



Vektor Dokumen

Setiap dokumen direpresentasikan sebagai deretan angka sesuai kamus yang sudah dibuat.



Vektor Query

Query pengguna (misalnya "laptop gaming") juga dikonversi menjadi vektor dalam ruang yang sama.



Bandingkan & Urutkan

Hitung Cosine Similarity antara vektor query dan setiap dokumen. Urutkan hasil dari skor tertinggi.



Masalah VSM Polos: Kasus Jus Alpukat

Simulasi GoFood

Kamu mencari "**jus alpukat**". Dua warung muncul:

Warung A: "*alpukat alpukat alpukat susu gula*"

Warung B: "*jus alpukat segar tanpa gula*"

Dengan VSM biasa (hitung frekuensi kata mentah), **Warung A menang** — karena "alpukat" disebut 3 kali!

Mengapa Ini Salah?

Padahal kamu ingin Warung B. Kenapa? Karena kata "**jus**" dan "**segar**" jauh lebih informatif dan relevan dibanding kata "alpukat" yang diulang-ulang.

Pengulangan kata yang sama tidak otomatis berarti dokumen itu lebih relevan. Kita perlu memberikan **bobot yang lebih cerdas** kepada setiap kata.

Solusinya: Gunakan pembobotan **TF-IDF** — kombinasi frekuensi dan keunikan kata.

TF: Seberapa Sering Kata Itu Disebut?

Definisi

Term Frequency (TF) mengukur seberapa sering sebuah kata muncul dalam satu dokumen, dinormalisasi terhadap panjang dokumen tersebut.

Rumus

$TF = \frac{\text{Jumlah kemunculan kata}}{\text{Total kata di dokumen}}$

Fungsi

TF menghargai kata yang sering diulang sebagai indikator topik utama dokumen. Semakin sering sebuah kata disebut, semakin besar pengaruhnya terhadap skor dokumen tersebut.

Contoh: Chat Grup Keluarga

📄 "besok besok besok kumpul kumpul di rumah"

Total kata: 6

TF("besok")

$3 \div 6 = 0.5$

Ini topik utamanya!

TF("kumpul")

$2 \div 6 = 0.33$

Topik kedua

TF("di")

$1 \div 6 = 0.17$

Kurang penting

IDF: Seberapa Langka Katanya?

Definisi

Inverse Document Frequency (IDF) mengukur seberapa "spesial" atau langka sebuah kata di seluruh koleksi dokumen. Kata yang muncul di mana-mana dianggap tidak informatif.

Rumus

$IDF = \log\left(\frac{\text{Total Dokumen}}{\text{Dokumen yang mengandung kata}}\right)$

Fungsi $IDF = \log\left(\frac{\text{Total Dokumen}}{\text{Dokumen yang mengandung kata}}\right)$

IDF "menghukum" kata-kata umum seperti "*yang*", "*di*", "*dan*" yang muncul di hampir semua dokumen – karena kata-kata itu tidak membantu membedakan dokumen satu dengan lainnya.

Contoh: Analisis Chat

Kata "yang"

Ada di **1.000** dari **1.000** chat

$IDF = \log(1000/1000) = 0$

→ Sampah! Abaikan saja.

Kata "lamaran"

Hanya ada di **1** dari **1.000** chat

$IDF = \log(1000/1) = 3.0$

→ WAH, ini PENTING!

Kata "ulang tahun"

Ada di **20** dari **1.000** chat

$IDF = \log(1000/20) = 1.7$

→ Cukup penting.

TF-IDF: Gabungan Otaknya

TF-IDF menggabungkan dua dimensi penilaian kata: seberapa sering kata itu muncul di dokumen ini, **dan** seberapa langka kata itu di seluruh koleksi.

TF-IDF = TF × IDF

Studi Kasus: Tokopedia

Query: "sepatu lari nike"

Produk	Deskripsi	TF	IDF	Skor
Produk A	"sepatu sepatu murah"	Tinggi	Rendah	Rendah
Produk B	"sepatu lari nike ori"	Sedang	Tinggi	TINGGI ✓

Mengapa Produk B Menang?

Produk A mengulang kata "sepatu" – kata yang ada di jutaan produk lain. IDF-nya rendah karena tidak langka.

Produk B mengandung "lari" dan "nike" – kata-kata yang lebih spesifik dan langka. IDF-nya tinggi sehingga skor TF-IDF jauh lebih besar.

Inilah alasan mengapa produk yang *benar-benar kamu cari* muncul di halaman pertama hasil pencarian.

Contoh Hitung Manual: Cari Kontak HP

Koleksi Kontak

D1

Budi Kantor Jakarta

D2

Budi Teman SMA

D3

Santi Kantor Bandung

Query: "Budi Kantor"

Langkah: Bangun vektor TF-IDF untuk setiap kontak → Hitung Cosine Similarity dengan vektor query.

Hasil Perhitungan

0.9

D1 – Budi Kantor Jakarta

Mengandung KEDUA kata query → skor tertinggi, muncul paling atas.

0.4

D2 – Budi Teman SMA

Hanya mengandung "Budi" → skor sedang.

0.3

D3 – Santi Kantor Bandung

Hanya mengandung "Kantor" → skor terendah.



Catatan : Ajak 1 mahasiswa maju untuk menghitung bersama langkah-langkah di atas secara manual di papan tulis.



Praktikum 2: Bikin Mesin Pencari Lagu

Spesifikasi Tugas

- **Koleksi:** 5 lirik lagu Indonesia pilihan kamu
- **Query:** "cinta patah hati"
- **Output:** Urutan lagu dari yang paling "patah hati" berdasarkan skor TF-IDF + Cosine
- **Tools:** Google Colab
- **Waktu:** 20 menit

Kode Inti (scikit-learn)

```
from sklearn.feature_extraction.text
import TfidfVectorizer
from sklearn.metrics.pairwise import
cosine_similarity
```

```
# Koleksi lirik lagu
daftar_lirik = [
    "cinta ini membara...",
    "patah hati lagi...",
    # tambahkan 3 lagu lagi
]
```

```
vectorizer = TfidfVectorizer()
X =
vectorizer.fit_transform(daftar_liri
k)
```

```
query = ["cinta patah hati"]
skor = cosine_similarity(
    vectorizer.transform(query), X
)
```

```
print(skor)
```

Rangkuman: Analogi Dapur

Semua model yang kita pelajari hari ini bisa dipahami melalui satu analogi sederhana: **proses memasak**. Setiap komponen memiliki peran yang berbeda dalam menghasilkan "hidangan" yang sempurna.



Boolean = Saringan

Bahan lolos atau tidak. Tidak ada gradasi — hanya 1 atau 0. Sederhana tapi kasar.



TF = Kadar Garam

"Garamnya kebanyakan" — kata yang sering disebut mendapat bobot lebih tinggi dalam dokumen.



IDF = Bumbu Rahasia

Langka dan penting. Tidak semua masakan pakai bumbu ini — itulah yang bikin spesial.



TF-IDF = Resep Terbaik

Gabungan "sering disebut" dan "bumbu rahasia" — menghasilkan skor relevansi yang akurat.



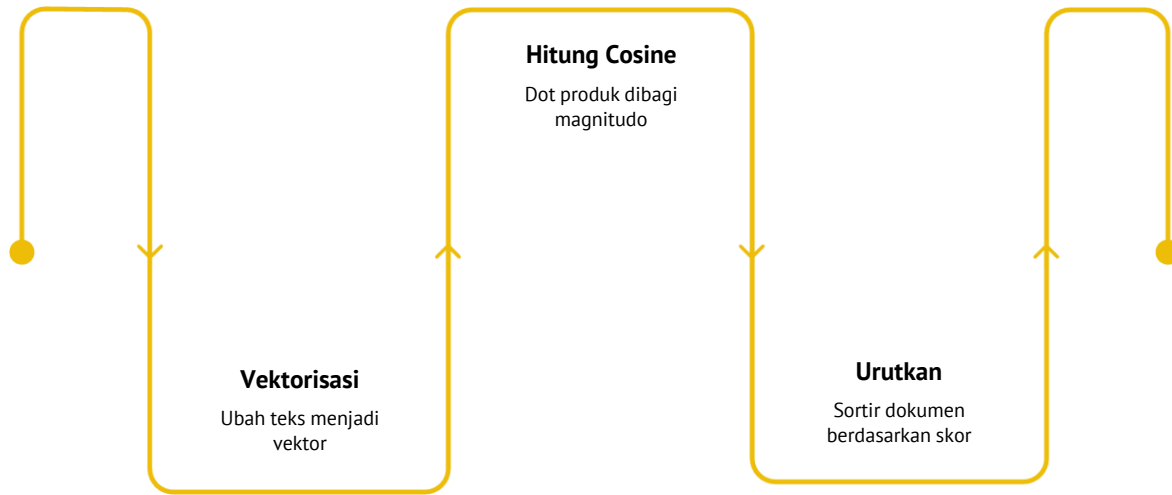
VSM = Uji Rasa

"Mirip rasanya" — bandingkan dua masakan berdasarkan sudut vektornya di ruang fitur.

Perbandingan Lengkap: Boolean vs VSM + TF-IDF

Dimensi	Boolean	VSM + TF-IDF
Output	Cocok / Tidak Cocok (0 atau 1)	Skor relevansi kontinu (0.0 – 1.0)
Ranking	✗ Tidak ada	✓ Terurut berdasarkan skor
Toleransi variasi bahasa	✗ Sangat kaku	✓ Lebih fleksibel
Kompleksitas bagi pengguna	Tinggi – harus tahu operator	Rendah – query natural
Cocok untuk	Database hukum, filter e-commerce	Mesin pencari web, rekomendasi
Contoh aplikasi	Filter Shopee, LexisNexis	Google, Netflix, Tokopedia

Visualisasi: Bagaimana Cosine Similarity Bekerja



Intuisi Geometris

Bayangkan setiap dokumen sebagai **panah (vektor)** yang menunjuk ke arah tertentu di ruang multidimensi. Query pengguna juga merupakan sebuah panah.

- Sudut kecil antara dua vektor → **sangat mirip**
- Sudut 90° → **tidak ada kemiripan**
- Arah yang persis sama → **identik (skor = 1)**

Cosine Similarity tidak peduli panjang vektor – hanya **arahnya** yang penting. Ini membuat dokumen panjang dan pendek bisa dibandingkan secara adil.

Koneksi ke Dunia Nyata

VSM dan TF-IDF bukan sekadar teori kuliah – teknologi ini ada di balik aplikasi yang kamu gunakan setiap hari.



Google Search

TF-IDF menjadi fondasi awal ranking Google sebelum dikembangkan dengan PageRank dan machine learning. Logika dasarnya masih sama: kata langka yang relevan = dokumen berkualitas tinggi.



Netflix Recommendations

Setiap film dan pengguna direpresentasikan sebagai vektor preferensi. Cosine similarity digunakan untuk menemukan film yang "paling dekat" dengan selera tontonanmu.



Spam Filter Email

Email spam memiliki TF-IDF yang tinggi untuk kata-kata seperti "gratis", "menang", "klik di sini" – berbeda dari email normal. Filter mengenali pola vektor ini secara otomatis.



Tokopedia / Shopee Search

Saat kamu mengetik query di kolom pencarian marketplace, TF-IDF bekerja di balik layar untuk memastikan produk paling relevan muncul di halaman pertama hasilnya.

Tugas Rumah: Eksplorasi VSM pada Skripsi

Instruksi Lengkap

1. Ambil **10 judul skripsi** dari Program Studi Teknik Komputer (bisa dari repository kampus)
2. Implementasikan **VSM dengan TF-IDF** menggunakan Python dan scikit-learn
3. Jalankan **3 query berbeda**: "AI", "Jaringan", "IoT"
4. Screenshot **3 hasil teratas** beserta skor cosine similarity untuk masing-masing query
5. Tulis **analisis 3 kalimat** per query: mengapa hasil tersebut muncul di posisi teratas?

Tips Pengerjaan

- Gunakan Google Colab – tidak perlu instalasi apapun
- Copy template kode dari praktikum hari ini sebagai starting point
- Pastikan judul skripsi dimasukkan dalam format list Python string
- Sertakan screenshot kode + output dalam laporan

Format Pengumpulan

PDF atau file Colab (.ipynb) dikirim via LMS sebelum pertemuan berikutnya. Cantumkan nama dan NIM di header notebook.



Preview Pertemuan 3: Mengukur Kualitas Mesin Pencari

Setelah bisa *membangun* mesin pencari, pertanyaan selanjutnya adalah: **bagaimana kita tahu apakah mesin pencari kita sudah bagus?**

1

Precision

Dari semua hasil yang dikembalikan mesin pencari, berapa persen yang benar-benar relevan? Mengukur **akurasi** hasil.

2

Recall

Dari semua dokumen relevan yang ada, berapa persen yang berhasil ditemukan? Mengukur **kelengkapan** hasil.

3

MAP

Mean Average Precision – metrik tunggal yang menggabungkan precision dan recall untuk mengevaluasi kualitas **ranking secara keseluruhan**.

📌 Teaser: Ada trade-off menarik antara Precision dan Recall – meningkatkan satu seringkali menurunkan yang lain. Kita akan bahas di P3!

Rangkuman Pertemuan 2

Boolean

Filter biner tanpa ranking. Cocok untuk pencarian terstruktur, tidak ideal untuk pengguna umum.

VSM

Ubah teks jadi vektor. Ukur kemiripan dengan Cosine Similarity. Hasilkan ranking yang bermakna.

TF-IDF

Bobot kata berdasarkan frekuensi (TF) $\times$ kelangkaan (IDF). Kata langka yang relevan = skor tinggi.

Python

Implementasi nyata dengan scikit-learn: `TfidfVectorizer + cosine_similarity` dalam ~10 baris kode.

"Mesin pencari yang baik bukan yang menemukan banyak hasil — tapi yang menemukan hasil **yang paling kamu butuhkan**, di posisi paling atas." 🎯

